NLP and Large Language Models

VL07: Post-Training I: Supervised Fine-Tuning

Course 8,862 | 6 ECTS | FS2026

Tuesday 14:15-16:00

Prof. Dr. Siegfried Handschuh

DS-NLP · Universität St. Gallen

Where We Are: The Course Navigator

Week	Topic	Notes
1	Introduction	✓
2	Tokenization & Data	✓
3	Transformer Architecture & Attention	✓
4	Training Fundamentals	✓
5	Scaling Laws	✓
6	Systems & Efficiency	✓
7	Post-Training I: SFT	*YOU ARE HERE*
8	Post-Training II: RLHF & DPO	
9	Inference & Decoding	
10	Evaluation & Benchmarking	
11	Applications: RAG & Agents	

Today's Milestone:

Your model is trained. Now you teach it to **follow instructions**.

Organisational

- **This lecture** is given by **Götz-Henrik** (Teaching Assistant)
- **Questions:** please contact Götz-Henrik directly
- **Next lecture:** 21 April, Prof. Handschuh, Topic: RLHF & DPO

Reminder: Midterm Presentation & 2x2 Split

Teams present their **pre-training progress** this week.

For post-training split your groups. (See excercise lecture slides from week 1)

Learning Objectives

1. Why Alignment?

Explain why a pre-trained LLM does **not** follow instructions, and what **alignment** fixes.

2. SFT Mechanics

Describe the SFT workflow: data format, **masked loss**, and how it teaches the assistant format.

3. Chat Templates

Explain why **structured templates** with special tokens are needed for multi-turn conversations.

4. LoRA

Explain how LoRA reduces trainable parameters by **100x** while preserving quality.

The Alignment Problem

The Base Model Problem [1]

Recall from VL04: pre-training optimizes $P(next\_token|context)$.
Your PikoGPT is excellent at **completing text**, but it does not know what «helpful» means.

Base Model (Pre-trained)

```
User:

«What is the capital of France?»
```

```
Model:

«What is the capital of Germany?»
«What is the capital of Italy?»
«What is the largest city in ...»
```

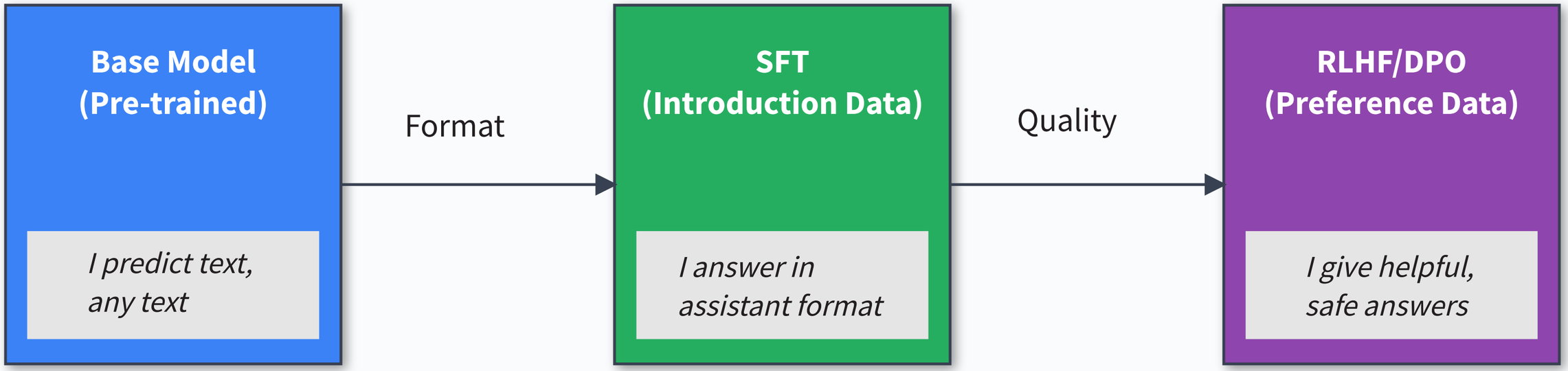
After SFT

```
User:

«What is the capital of France?»
```

```
Assistant:

«The capital of France is Paris.
It is also the largest city in France.»
```

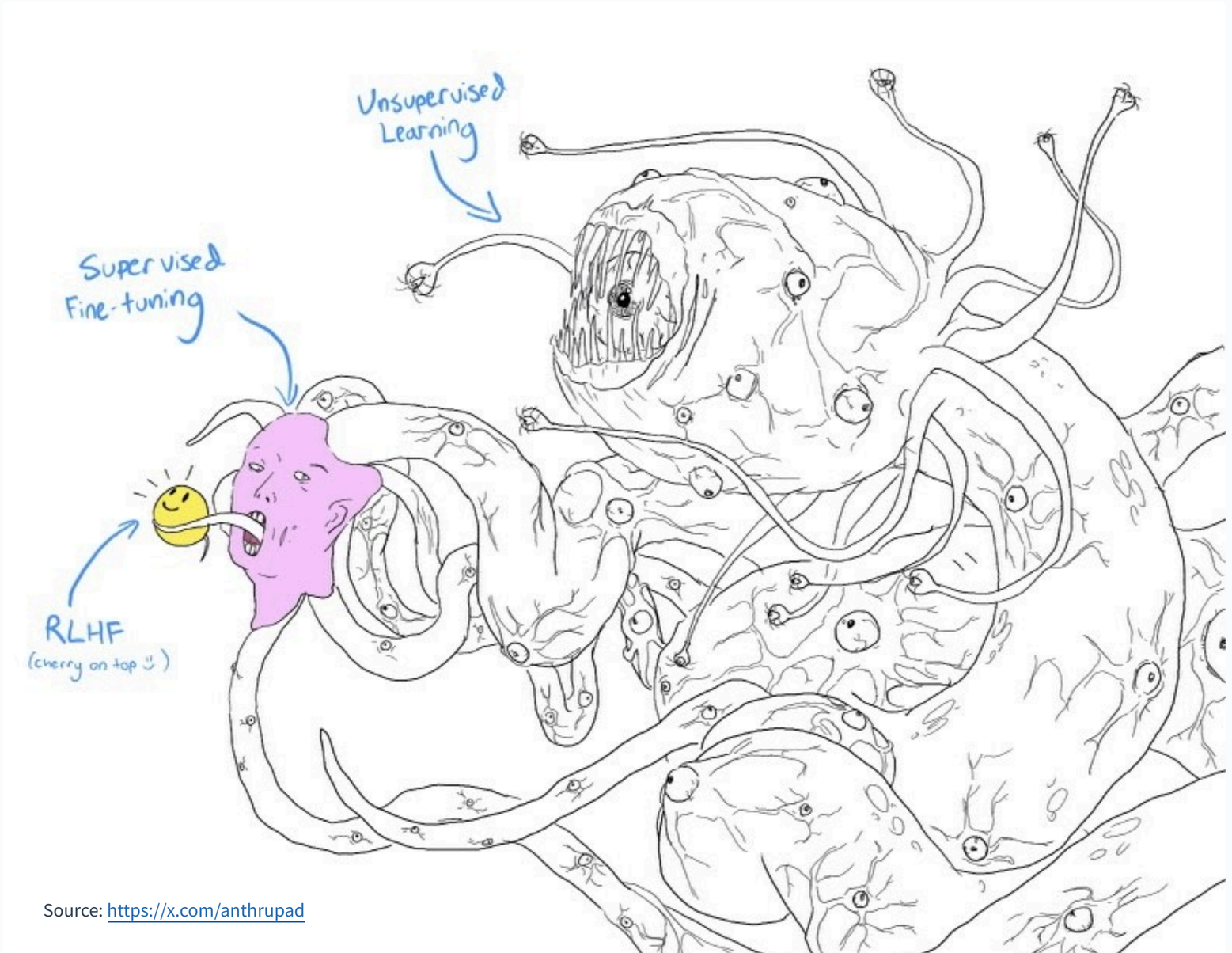
The base model continues.
SFT teaches it to answer.

The Alignment Pipeline [1]



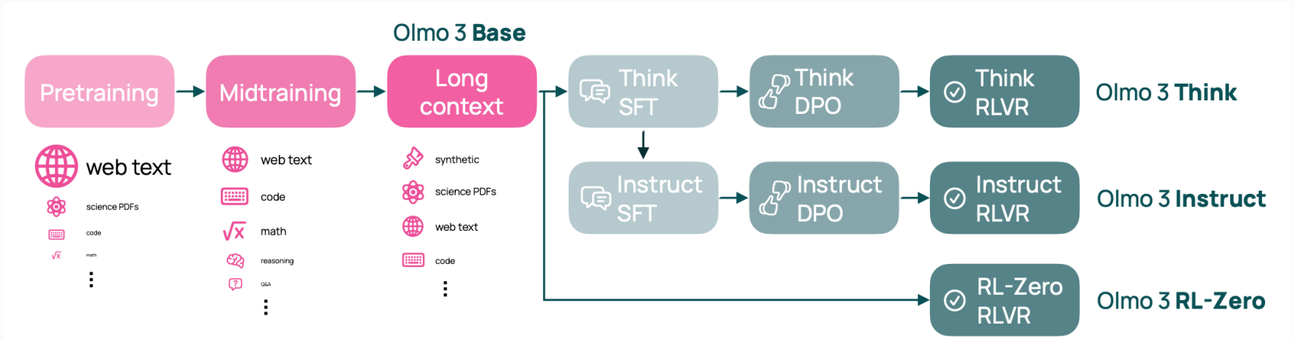
Each stage adds a new capability. SFT alone often gets you 80% of the way. InstructGPT: SFT improved human preference win rate from ~25% to ~65%.

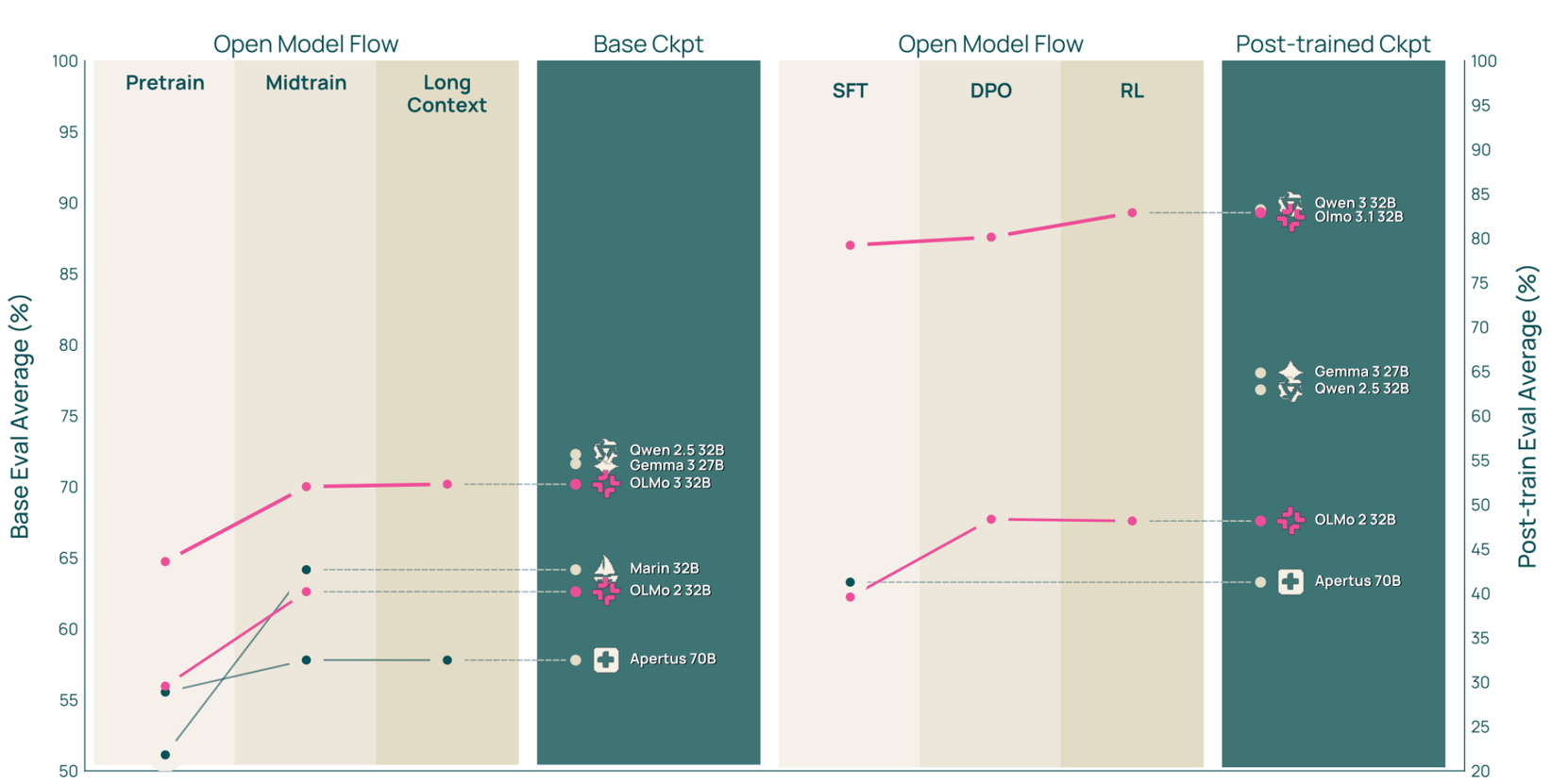
"Taming the Monster"



Source: https://x.com/anthrupad

Example: OLMO 3 Training Stages





https://arxiv.org/pdf/2512.13961

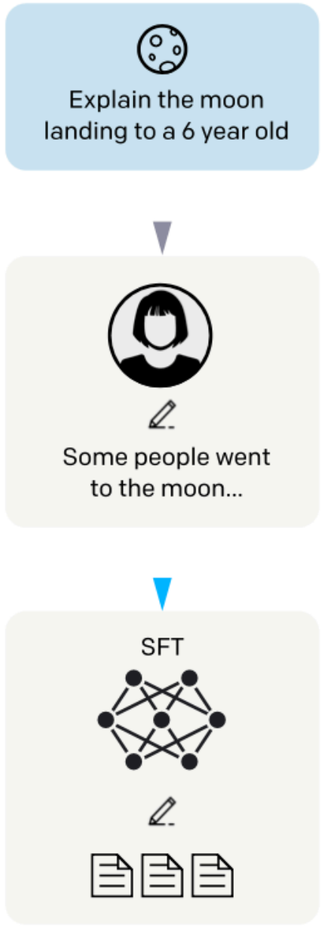
Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



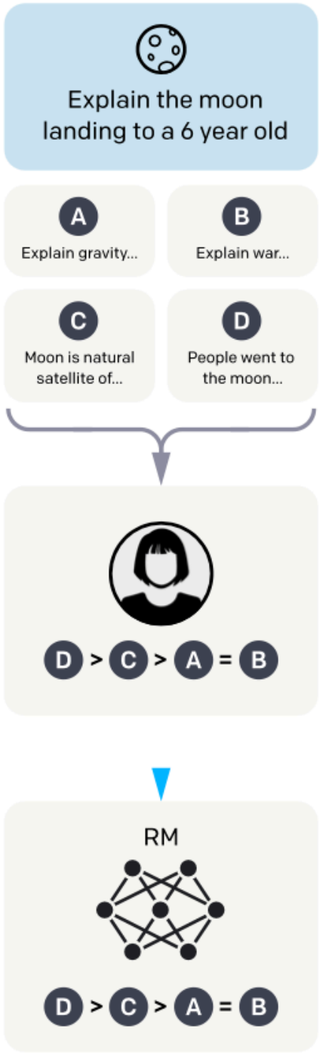
Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

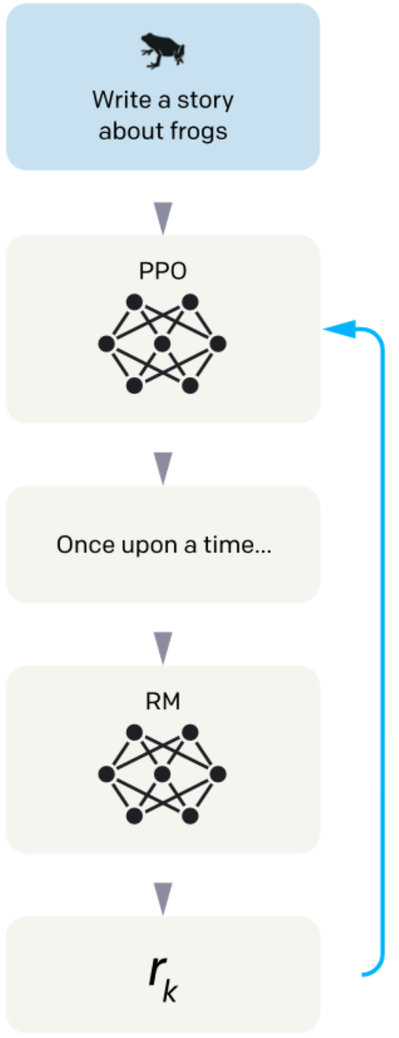
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



Alignment Training vs. Post-Training

Post-training:

- Everything you do after the base model has been pretrained.

Alignment Training:

- A subset of post-training focused on making the model behave according to human values, preferences, and intent.

Alignment is often the main goal of Post-training. But post-training can be more than alignment-training.

Take-Home Message: The Alignment Problem

Pre-training teaches language. Post-training teaches behavior.

Base Model

Predicts text. Completes anything. No roles, no safety.

+ SFT

Follows instructions. Answers in assistant format. Still no quality filter.

+ RLHF/DPO

Gives *good* answers. Refuses harmful requests. Honest about uncertainty.

Today: Stage 2 (SFT).
Next week (VL08): Stage 3 (RLHF/DPO).
Your PikoGPT lives in Stage 1.

Supervised Fine-Tuning (SFT) Mechanics

SFT: The Core Idea [1]

SFT = Supervised Fine-Tuning -> model fine-tuning on instruction-response pairs.

Input Data

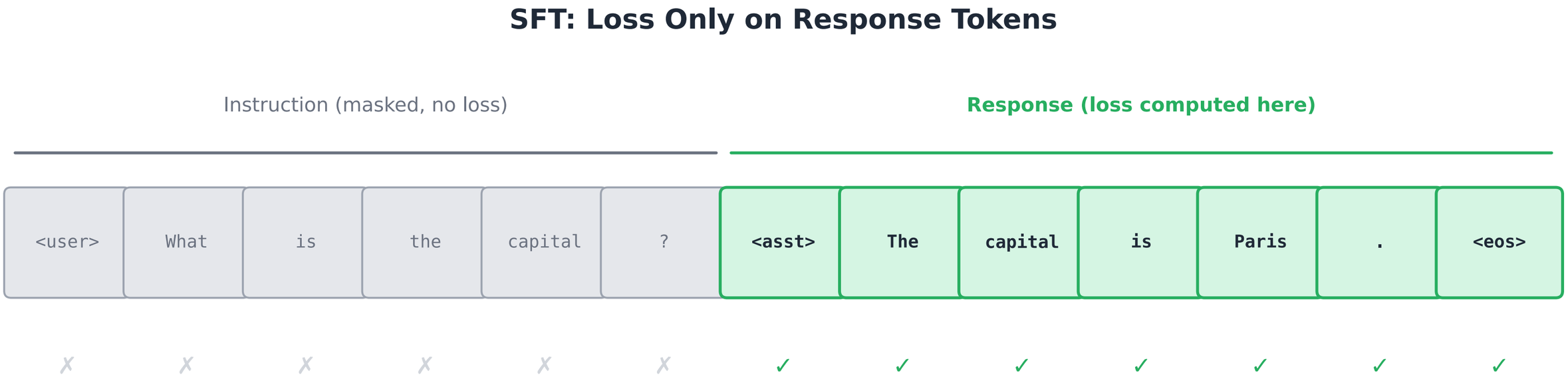
Pairs of (instruction, response) written by humans or generated by a stronger model.

Training Objective

Same CrossEntropy loss as pre-training, but **only on the response tokens**.

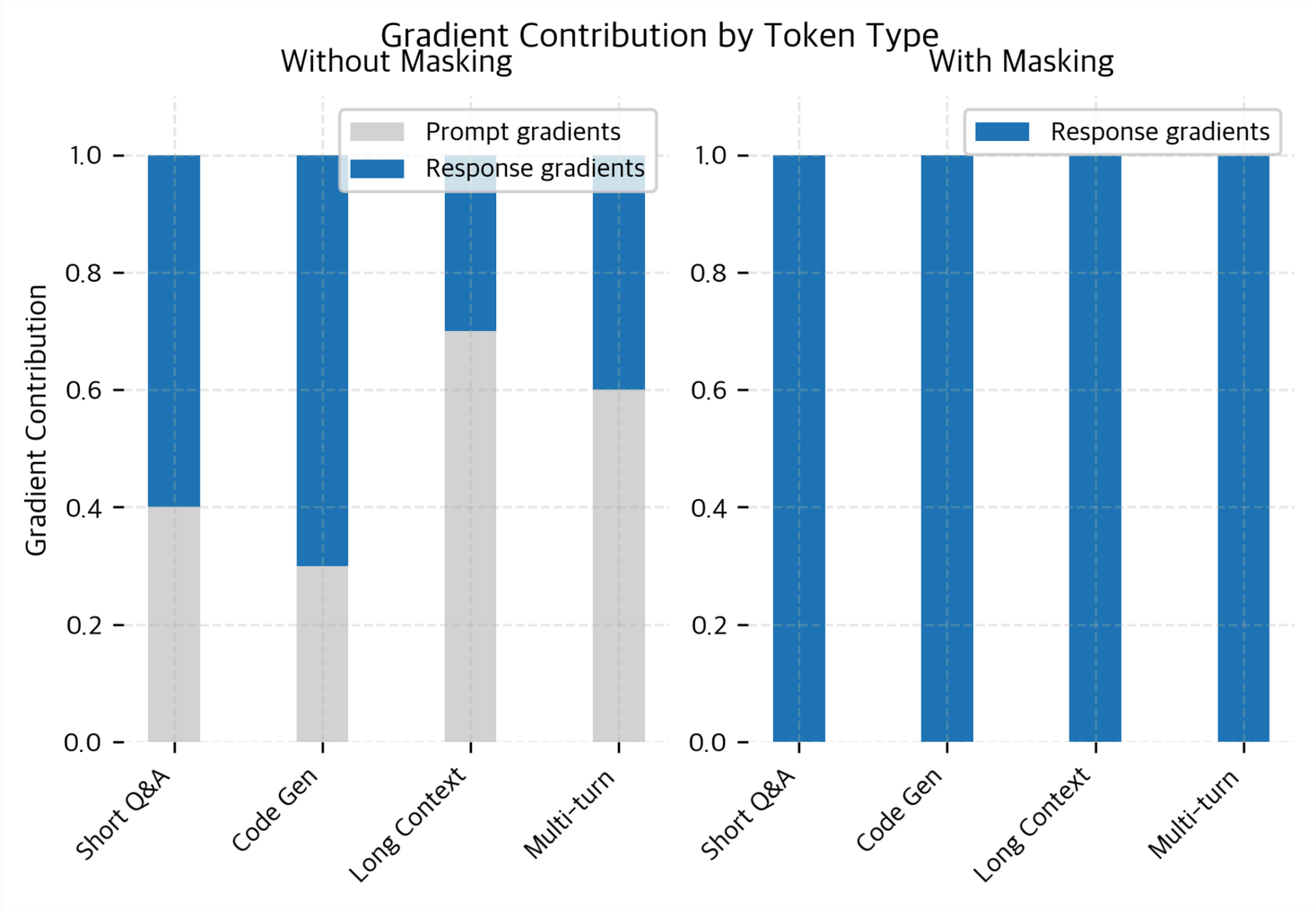
SFT teaches the model WHAT to say (assistant format), not just HOW to continue text.

The Masked Loss: Only Learn the Response [2]



The model sees the full sequence, but the loss is computed **only on response tokens**. Instruction tokens are masked with label = -100.

Token-Level Loss Masking: Detail View



SFT Data: What Does It Look Like? [3]

Instruction:

«Explain the concept of gravity to a 5-year-old.»

Response:

«You know how when you throw a ball up, it always comes back down? That is because the Earth pulls everything toward it. We call that pull gravity!»

Instruction:

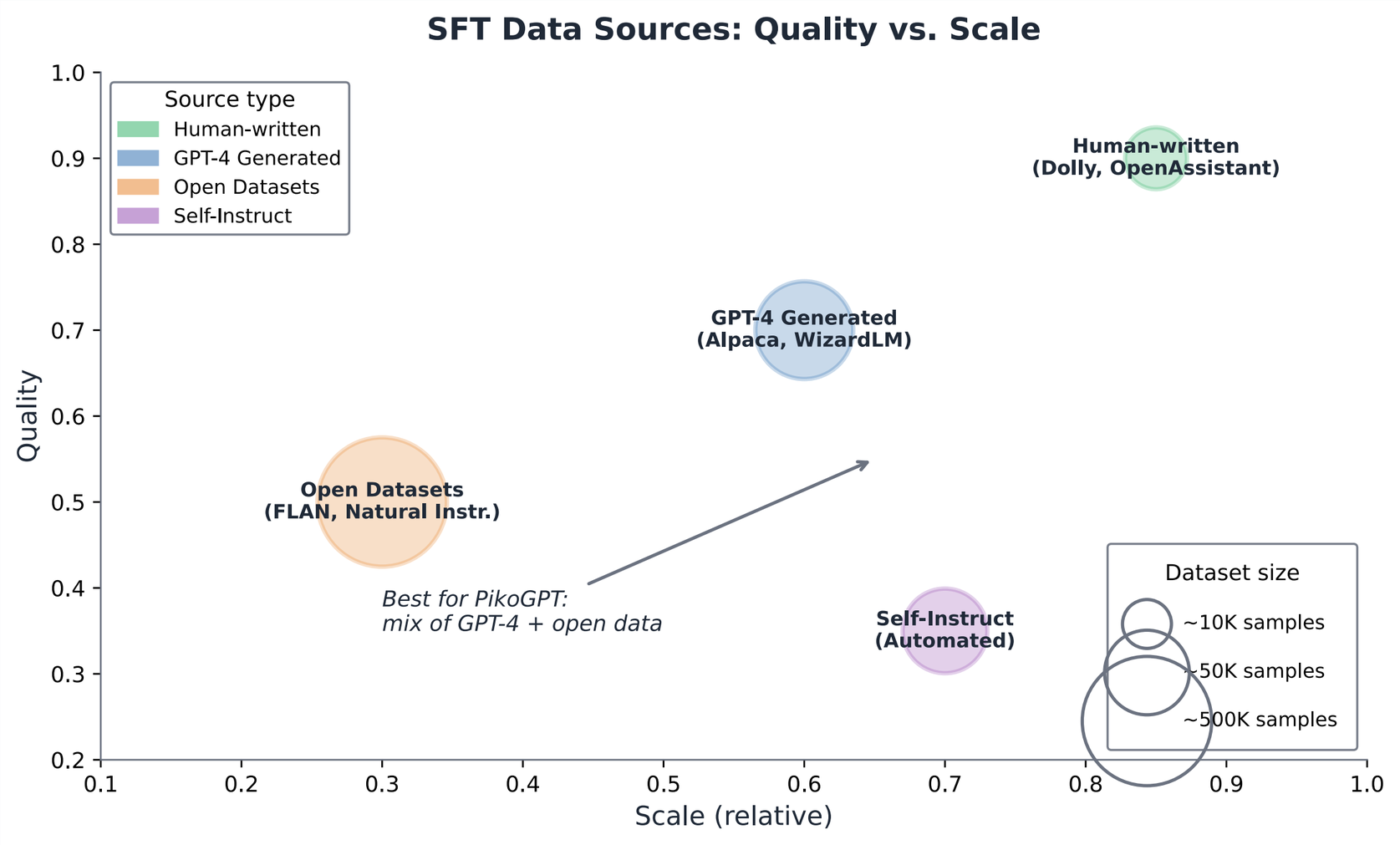
«Write a haiku about mountains.»

Response:

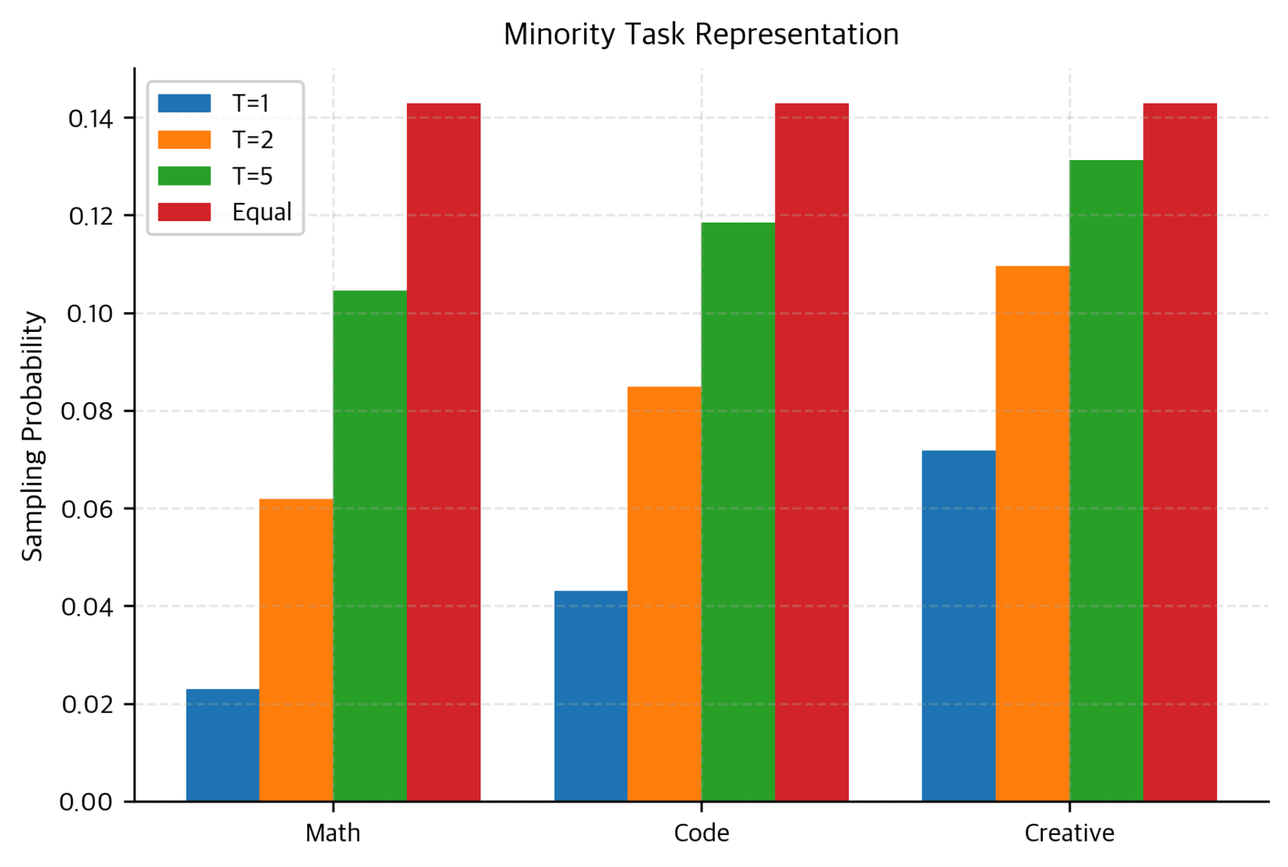
«Snow-capped peaks rise high
Silent guardians of earth
Touching winter clouds»

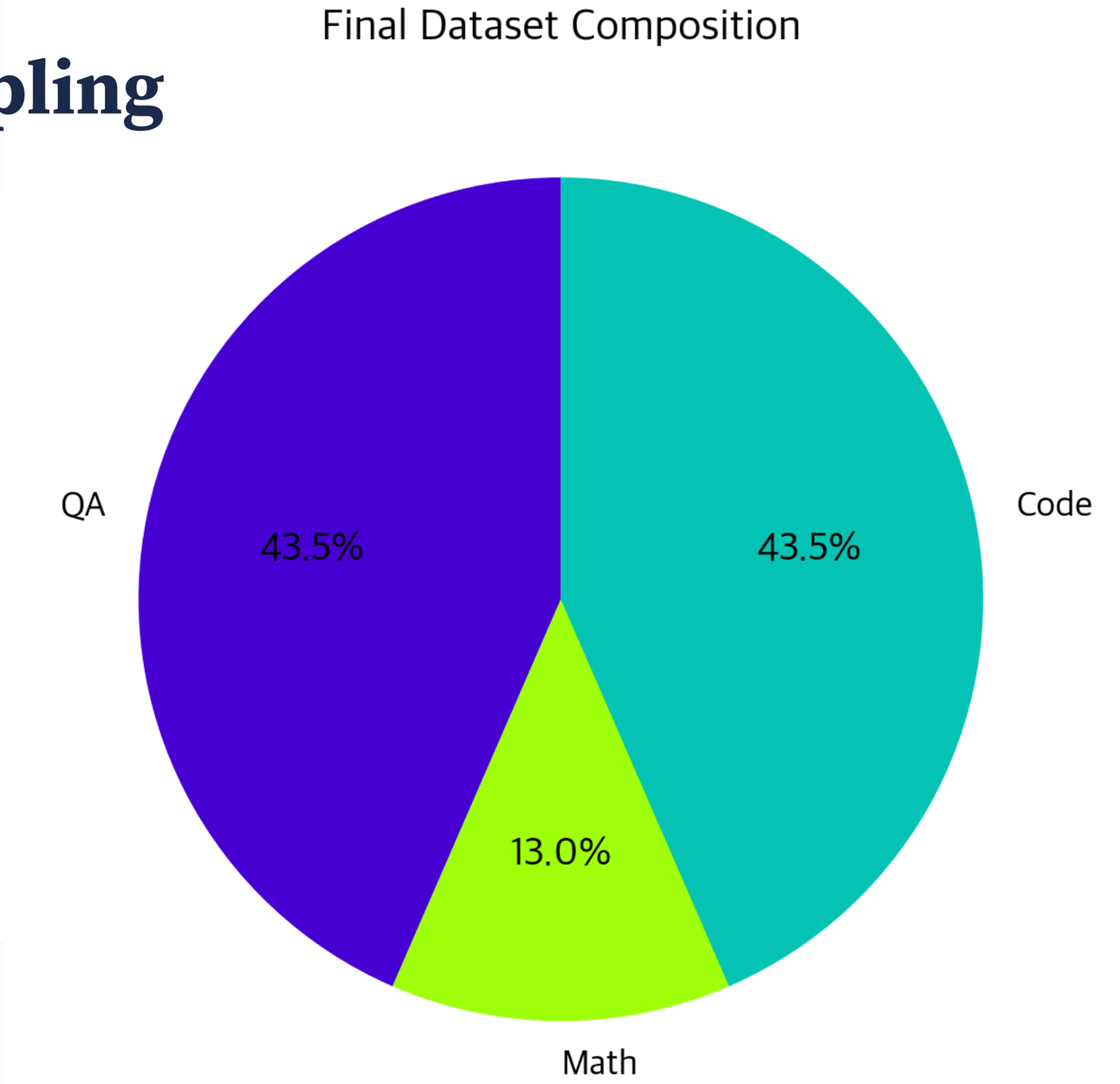
Typical datasets:
Alpaca (52K pairs, GPT-4 generated) [3],
Dolly (15K, human-written),
OpenAssistant (161K, community)

Data Sources: Quality vs. Scale [4]

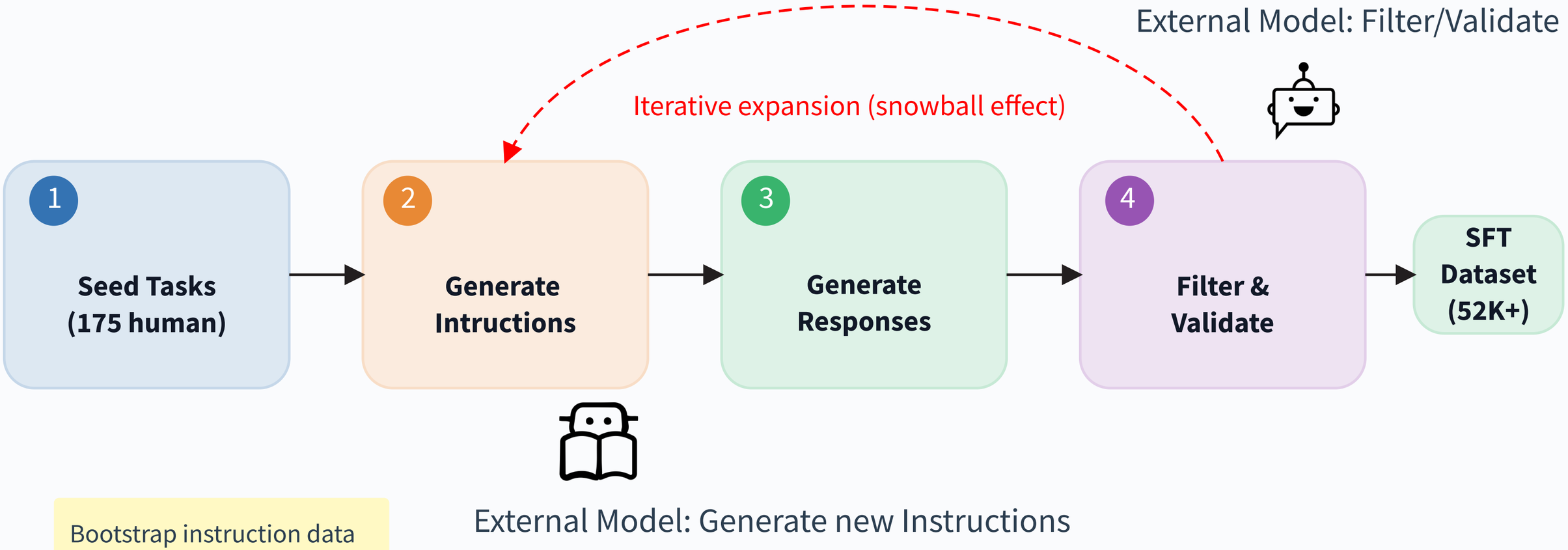


Data Mixing and Task Sampling



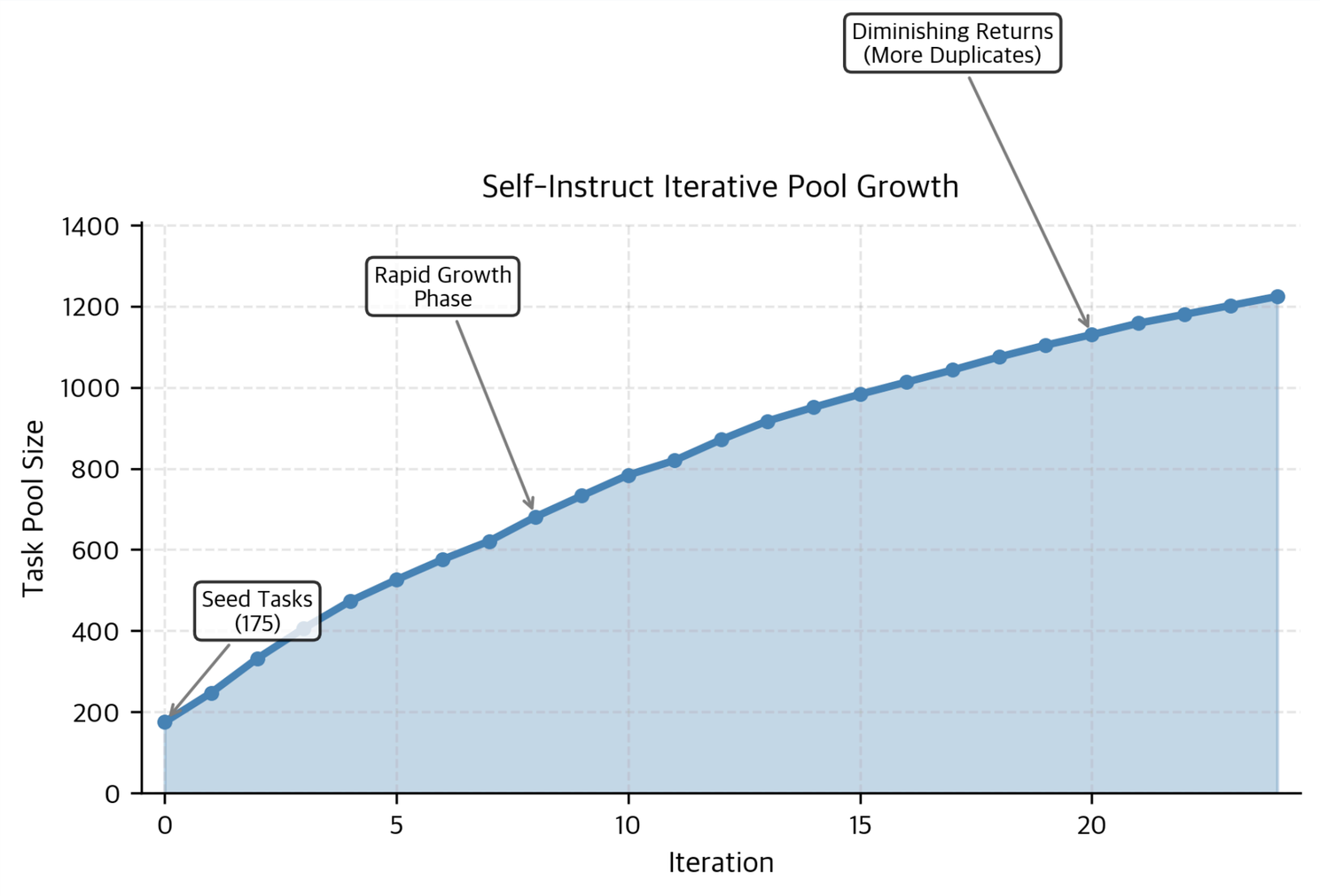


Example - Self-Instruct: Bootstrapping Data [4]



Bootstrap instruction data from a small see set using the LLM itself

Self-Instruct Pool Growth



Quick Poll: Your Data Strategy

Which data approach would you choose for PikoGPT SFT?

A. Human-written only (small, high quality)

B. Synthetic only (large, variable quality)

C. Mix of both

D. Skip SFT, focus on base model

Think About It: SFT Data (2 min)

Your team has a working PikoGPT (~<40M params). You want to add SFT. You have two options:

👩‍🔬 **Option A:** 500 instruction-response pairs, carefully written by your team

🤖 **Option B:** 50,000 pairs auto-generated by GPT-4

Which would you choose for your PikoGPT demo? Why?

SFT Limitations [1]

SFT teaches the **format**, but not the **quality**.

What SFT Can Do

- Follow instructions.
- Answer in assistant format.
- Stay on topic.

What SFT Cannot Do

- Distinguish good from great answers.
- Refuse harmful requests.
- Be calibrated.

This is why we need RLHF or DPO (VL08): explicit preference feedback on response quality.

Warning: Catastrophic Forgetting

What is CF?

- Model overrides learned Weights
- Overfitting to the alignment training data
- "forgetts" prior trained behaviour
- Reduces model robustness across tasks

What can we do to avoid it?

- Mix pretraining data with SFT data
- Use smaller learning rates
- Apply regularization (e.g. weight constraints)
- Early Stopping: stop training befor overfitting
- Use techniques like LoRA or parameter-efficient tuning

Goal: Strategically fine-tune models while preserving essential knowledge

Take-Home Message: SFT Mechanics

SFT = fine-tuning on instruction-response pairs with masked loss.
Simple but transformative.

- **Data:** instruction-response pairs. Quality > quantity.
- **Loss:** CrossEntropy on response tokens only (instruction masked).
- **Limitation:** teaches format, not quality. VL08 addresses this.
- **Catastrophic Forgetting:** avoid overtraining and "losing" your pretraining

Chat Templates & Special Tokens

Why Templates Matter

Without structure, the model cannot distinguish who said what.

❌ Without Template

```
«What is gravity?
Gravity is a force that pulls objects
toward each other. Can you explain more?»
```

Who said what? Where does the answer end?

✅ With Template

```
<user> What is gravity?
<assistant> Gravity is a force ...
<user> Can you explain more?
```

Clear roles. Clear boundaries.

Templates solve: role identification, turn boundaries, and context management.

Chat Template Structure [2]

Chat Template: Structured Multi-Turn Format

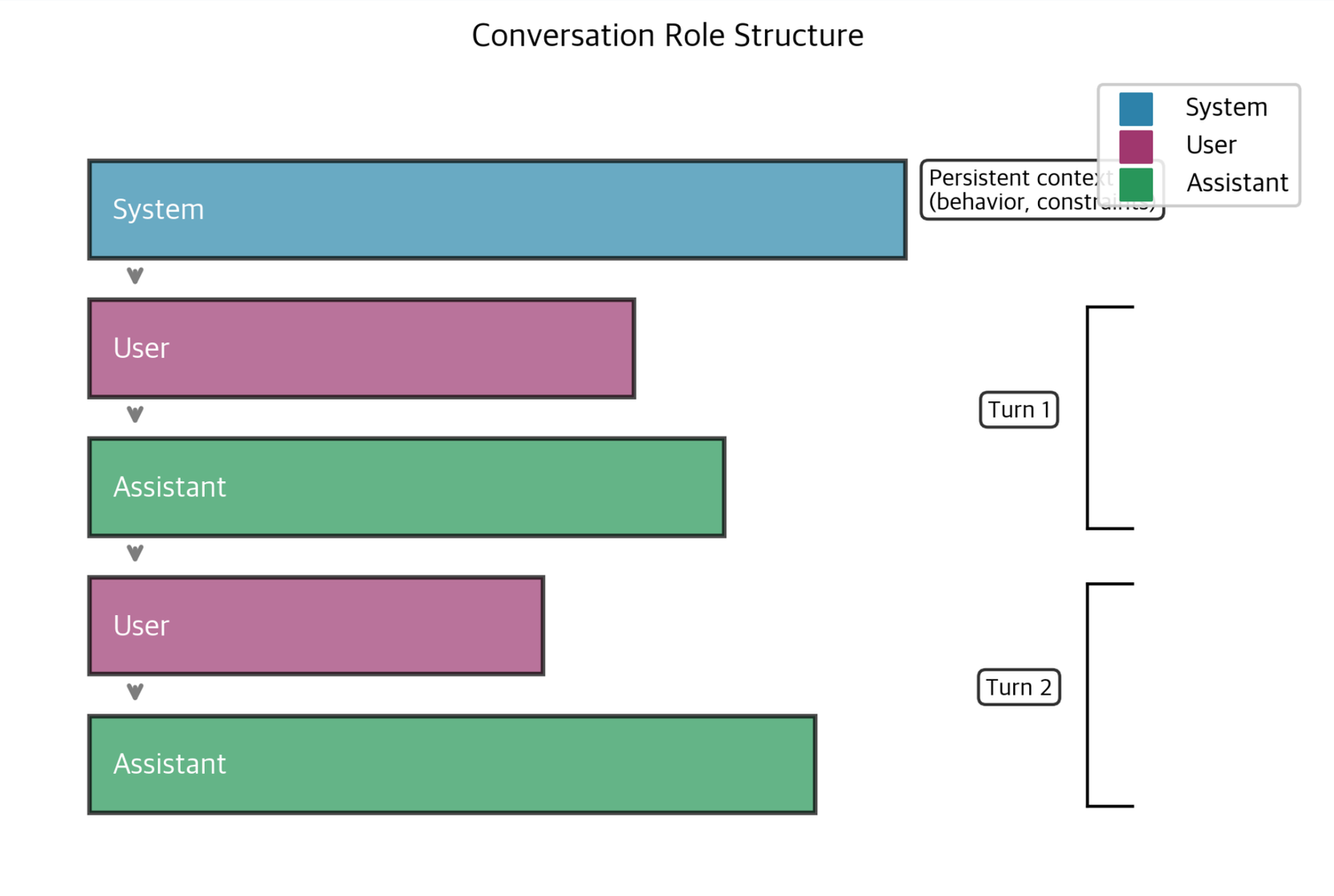
system	You are a helpful assistant.
user	What is the capital of France?
assistant	The capital of France is Paris.
user	And Germany?
assistant	The captial of Germany is Berlin.

Special tokens

`(<|im_start|>, <|im_end|>)`

separate each message

Multi-Turn Conversation Structure



Template Formats: ChatML & LLaMA-2

ChatML (OpenAI)

`<|im_start|> system`
You are helpful. `<|im_end|>`
`<|im_start|> user`
Hello! `<|im_end|>`
`<|im_start|> assistant`

Special tokens mark start/end of each role. Clean, extensible, widely adopted.

LLaMA-2 (Meta)

`<s>[INST] <<SYS>>`
You are helpful.
`<</SYS>>`
Hello! `[/INST]`

Uses `[INST]/[/INST]` tags with `<<SYS>>` block for system prompt.

Both require **new special tokens** added to the tokenizer vocabulary before training.

Template Formats: Alpaca & Recommendation

Alpaca (Stanford) [3]

Example instruction for task.

Instruction:
Hello!

Response:

Uses plain text markers only. No new special tokens needed.

Recommendation for PikoGPT:

- Alpaca format is the simplest choice. No tokenizer changes required.
- Uses Markdown known syntax.

Critical rule: the template used during **training** MUST match the template used during **inference**.

Mini-Check: SFT & Templates

1. In SFT, the loss is computed on _____ tokens only. The _____ tokens are masked.

2. A base model completes text. An SFT model _____ instructions.

3. Why is it critical that the chat template used in training matches the one used in inference?

LoRA: Efficient Fine-Tuning

The Problem: Full Fine-Tuning Is Expensive

Full Fine-Tuning

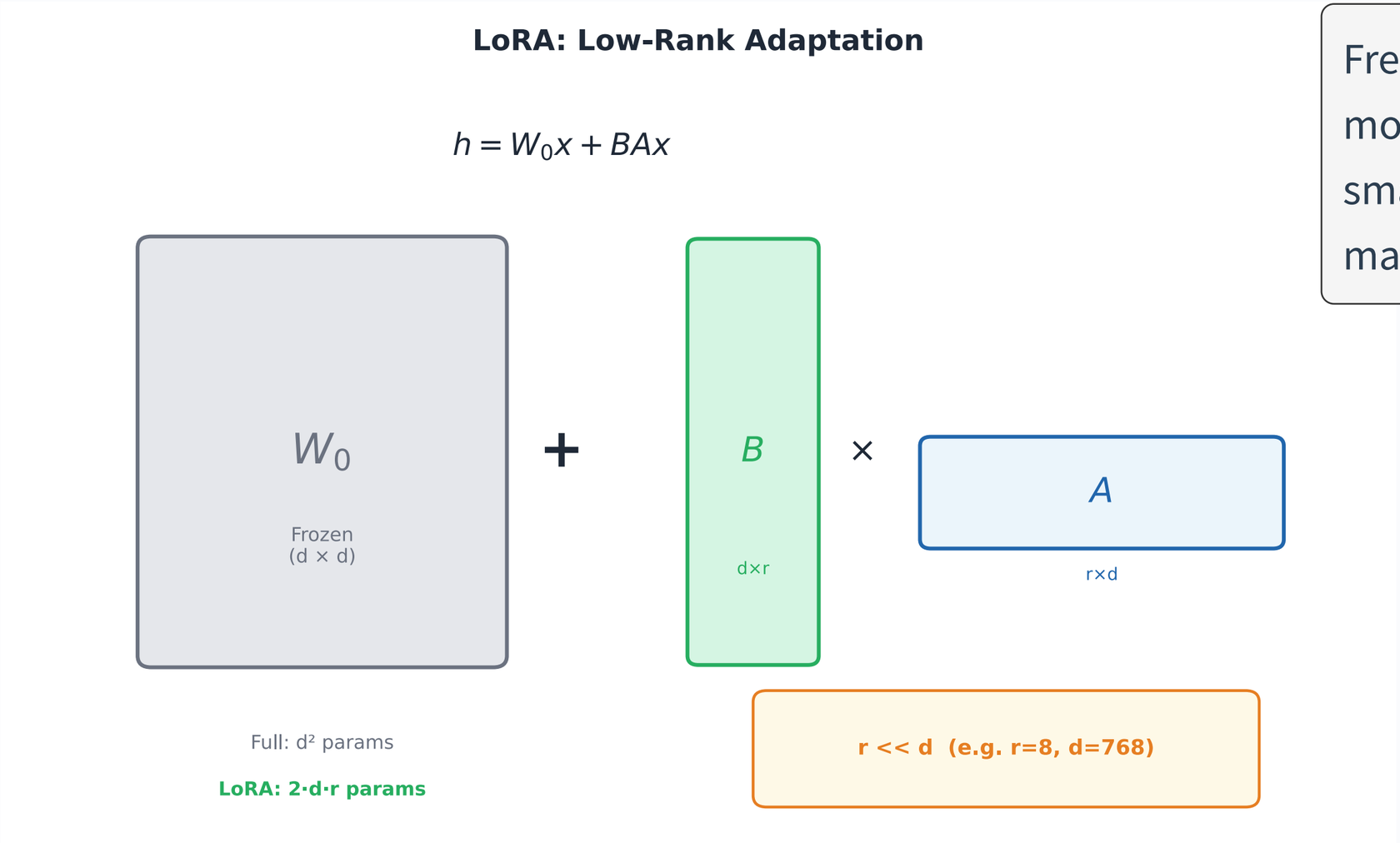
Update **all** parameters. For LLaMA-7B: 7 billion weights in FP16 = **14 GB** just for the model, plus optimizer states (2x) = **42 GB** minimum.

Parameter-Efficient (PEFT)

Update only a **small subset** of parameters. Freeze the base model. Add small trainable modules. **<1 GB** extra.

For PikoGPT (~40M): full fine-tuning fits on one GPU. For real LLMs: PEFT is essential to consumers to finetune models.

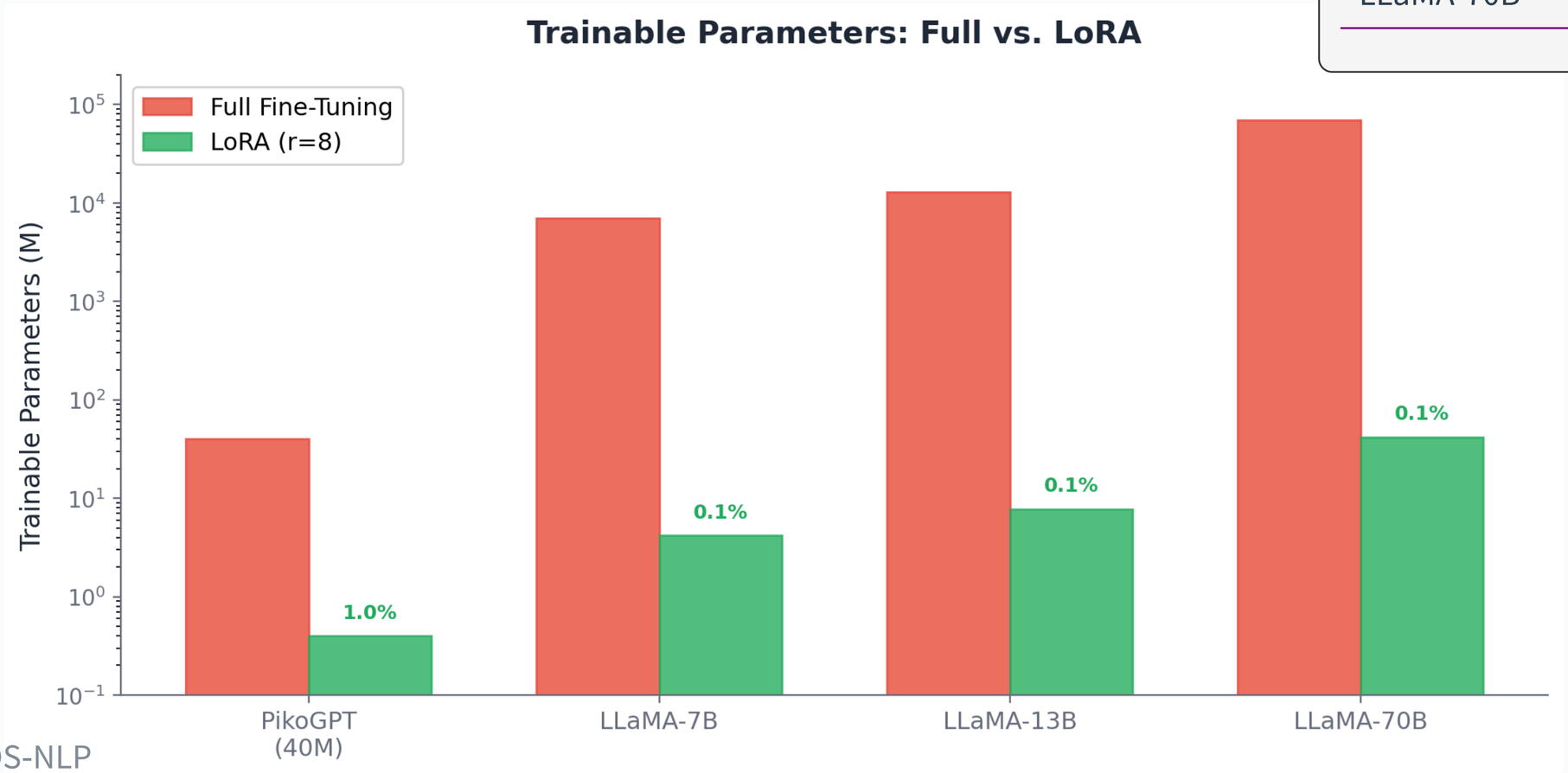
LoRA: The Key Idea [5]



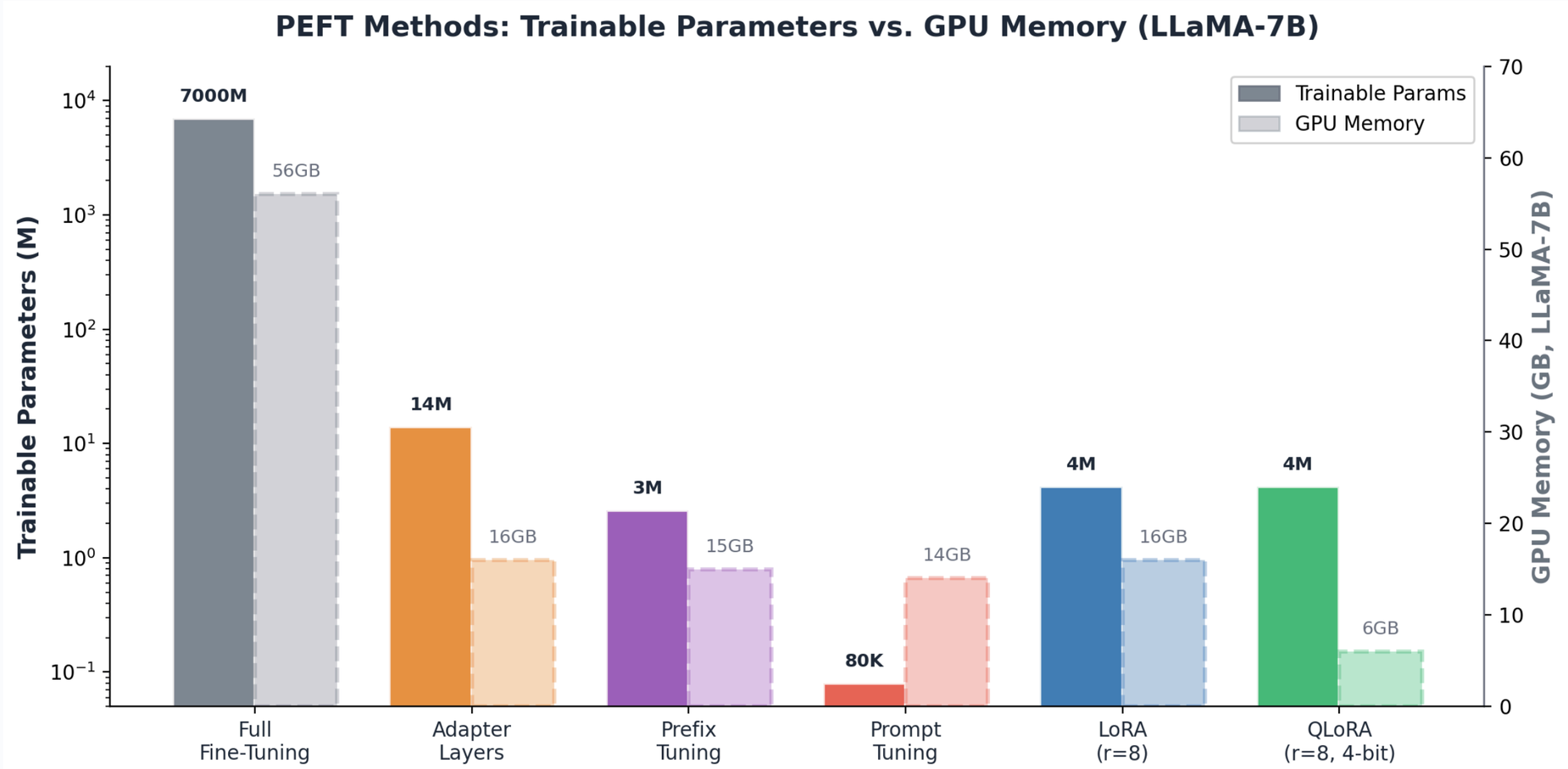
Freeze the base model. Train only small low-rank matrices.

How Big are the Savings? [5]

Model	Full FT Params	LoRA (r=8)	Savings
PikoGPT (~40M)	~40M	~400K	~100x
LLaMA-7B	7,000M	~4M	1,750x
LLaMA-70B	70,000M	~42M	1,667x

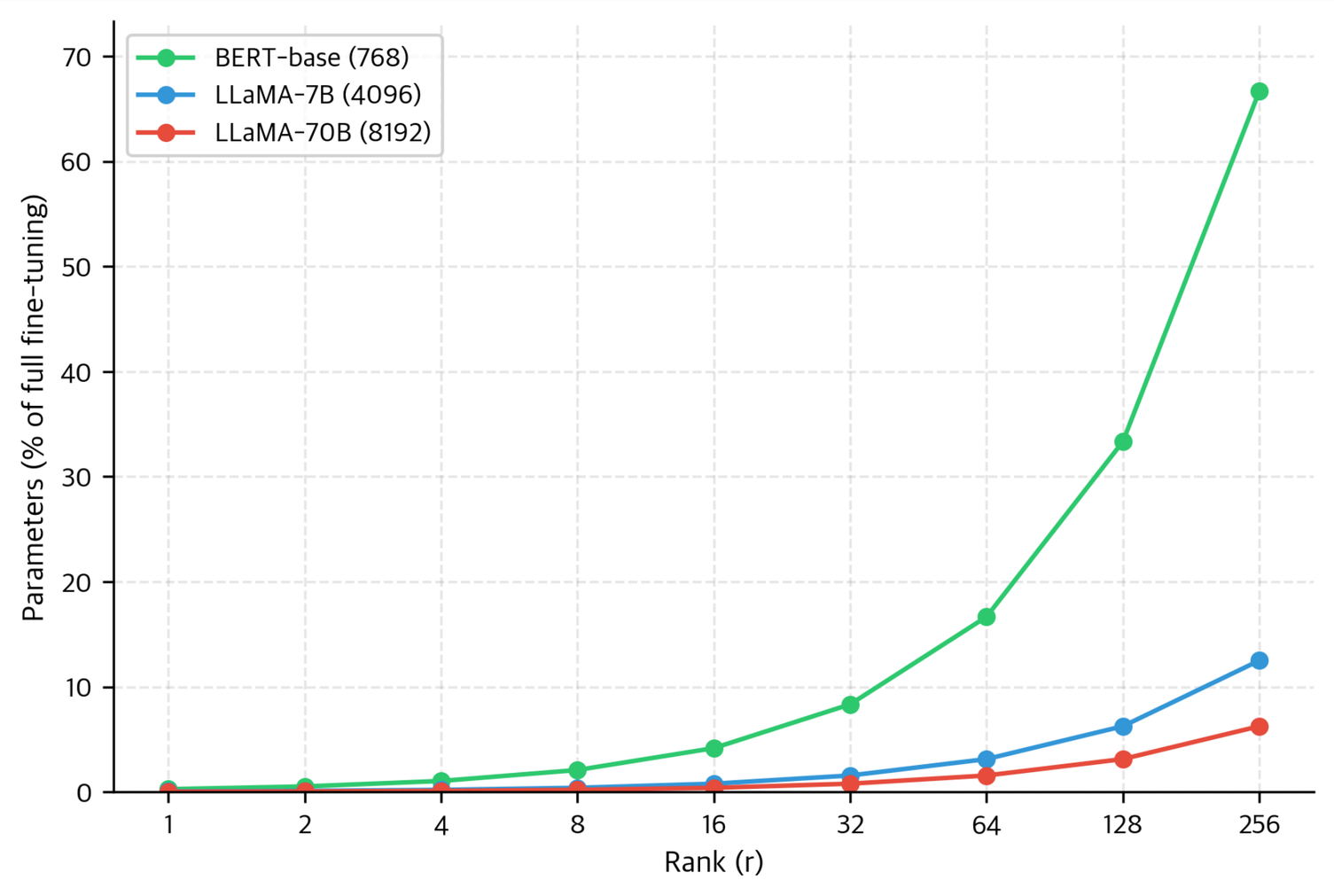


The PEFT Landscape: Beyond LoRA

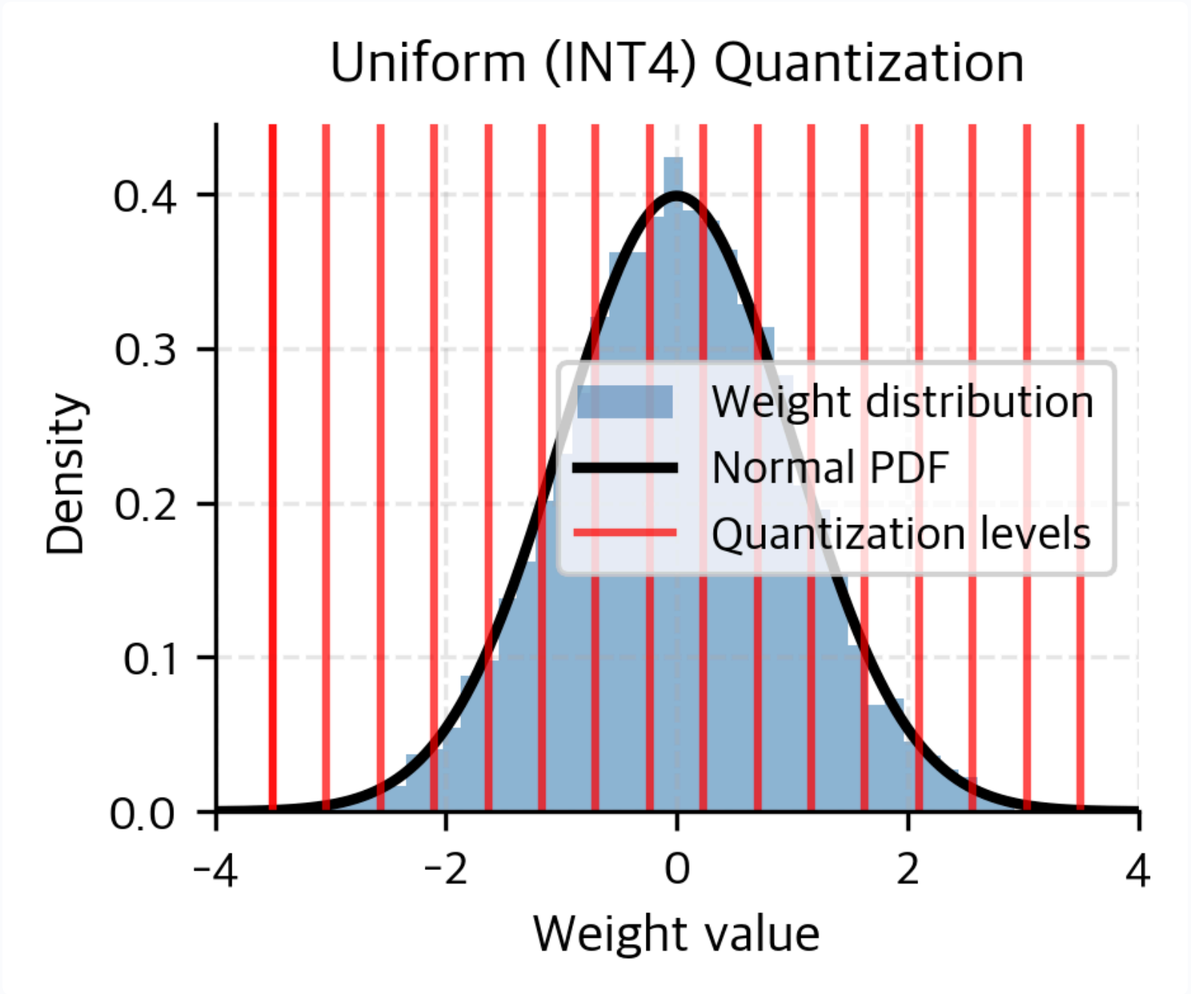


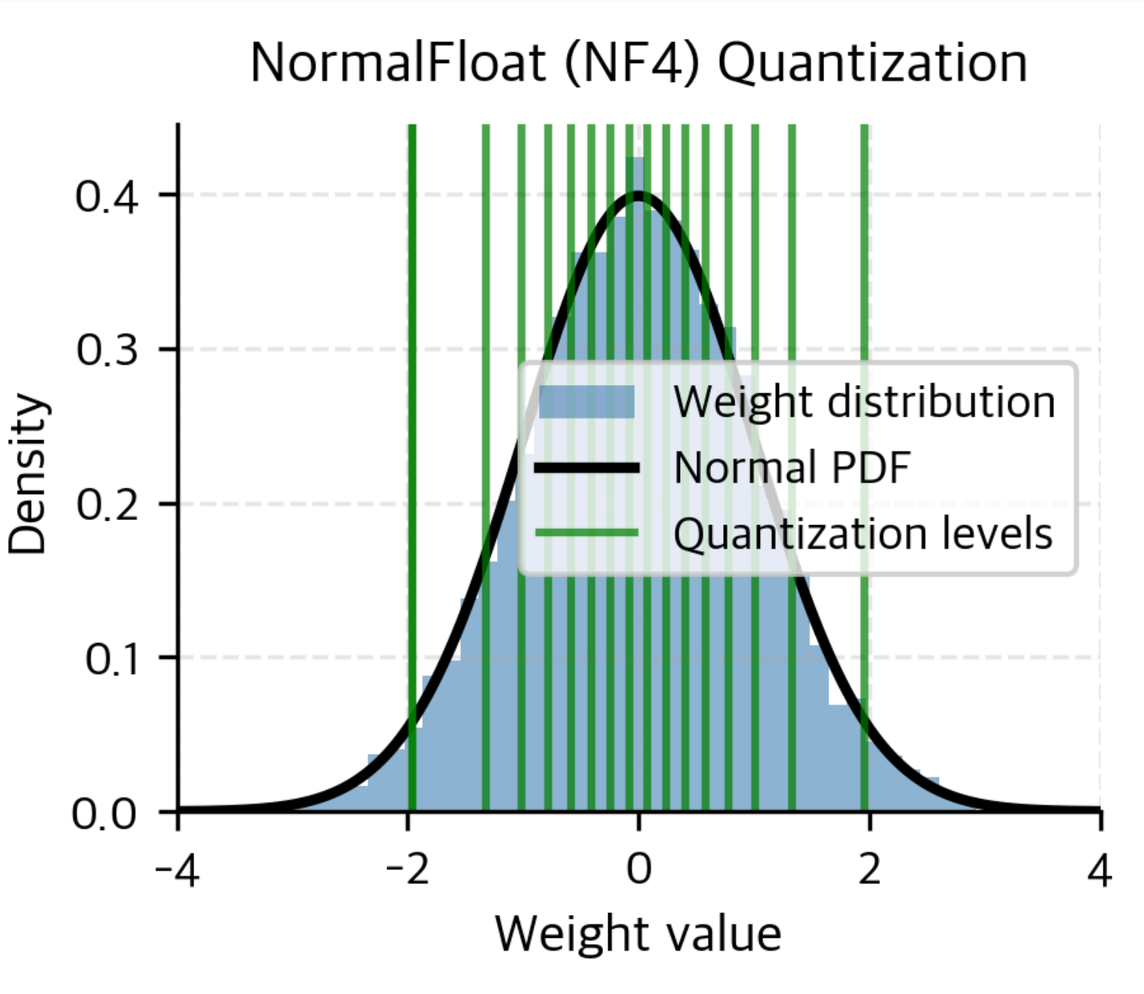
LoRA is most popular choice, but not the only one. **QLoRA** adds 4-bit quantization for even less memory. [6]

LoRA Rank Selection Guidance

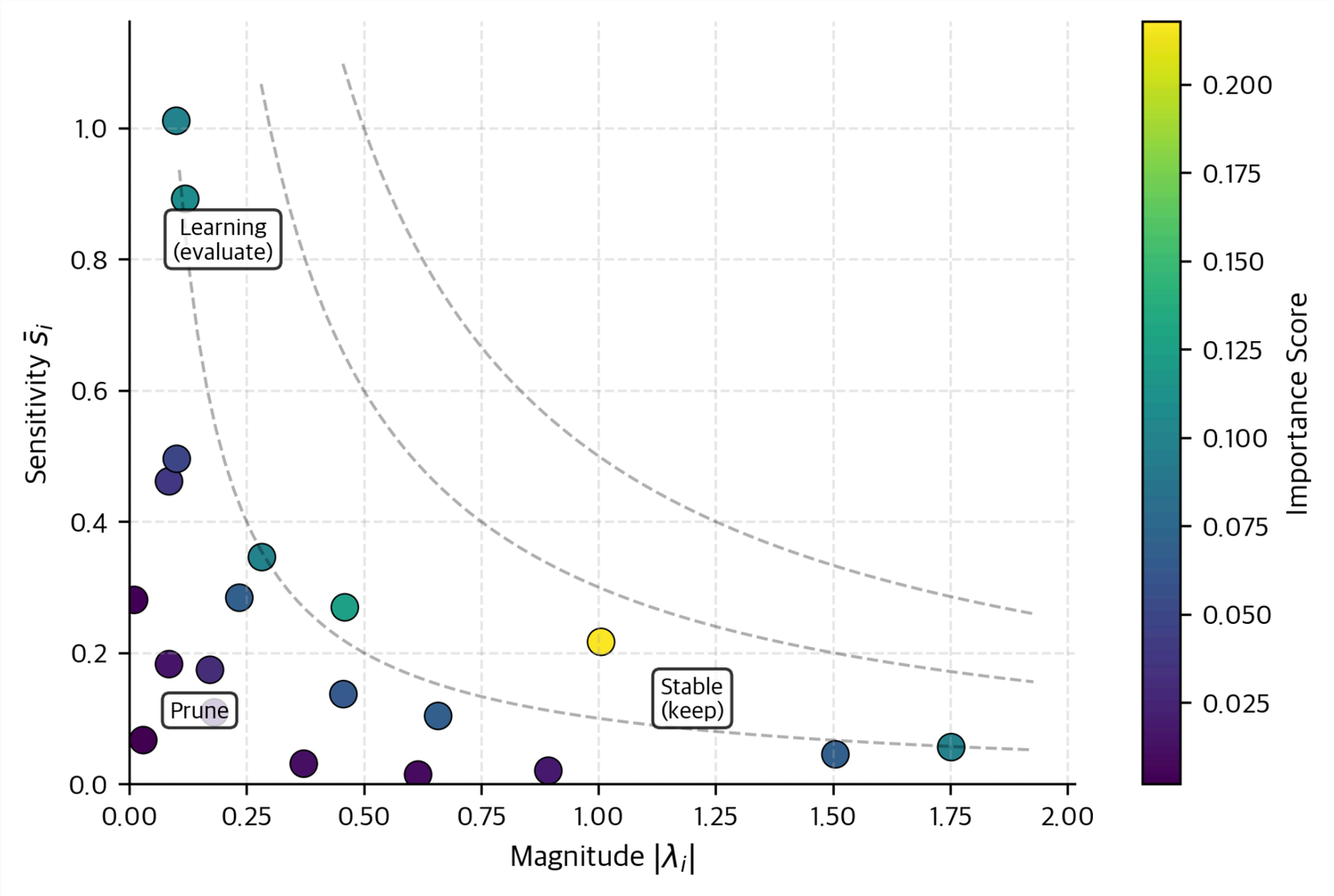


Excurse: QLoRA and NF4 Quantization





More Variants: AdaLoRA - Adaptive Rank Allocation



LoRA in Practice [5,6]

Hyperparameters

Rank r: 4, 8, 16

Alpha α: typically $2r$

Target: Q, K, V, O projections

Advantages

Same base model, multiple adapters. Merge at inference (no latency). Train on consumer GPUs.

Limitations

Not as expressive as full FT. Higher rank = closer to full FT. Not ideal for large domain shifts.

Rank selection rule of thumb:

r=8 is a strong default. Increase for complex tasks, decrease for simple format changes. QLoRA [6] adds 4-bit quantization for even less memory.

Discuss: Full Fine-Tuning vs. LoRA for PikoGPT (2 min)

Your 2x2 sub-group is planning the alignment phase. Consider:

1. PikoGPT has ~40M parameters. What would be a good rank r to fine-tune the model with LoRA?

2. You want to create your own OLMO model and you want to share it with a friend. How does LoRA help?

3. When would LoRA be a bad choice?

Hint: Think about the 2x2 split and what «unchanged base model» means.

Take-Home Message: LoRA

Freeze the base model. Train small low-rank adapters. Get up to 90%+ of full fine-tuning quality.

- **$W = W_0 + BA$**: rank $r << d$. Trainable params reduced by 100-1000x.
- **Practical:** same base model, multiple task adapters. Merge at inference.
- **Default:** r=8, apply to Q/K/V/O. QLoRA for even less memory.

Quick Check: The Full Picture

1. Pre-trained LLMs are good at _____ but bad at _____.

2. SFT uses _____ loss on _____ tokens only. The instruction is _____.

3. In LoRA, the trainable parameters are the matrices ___ and ___ with rank ___.

4. Why must the chat template be identical in training and inference?

These four questions map 1:1 to the four learning objectives.

Summary & Next Steps

Summary: From Base Model to Assistant

Alignment

Base models predict text, not follow instructions.

SFT

Fine-tune on instruction-response pairs. Masked loss.

Templates

Structured format with roles + special tokens.

LoRA

$W_0 + BA$. 100x fewer trainable params.

Next week (VL08): RLHF & DPO. How to go from «follows instructions» to «gives *good* answers».

For Your PikoGPT Project

Recommendations for PikoGPT:

- Use Alpaca[3] dataset for chat alignment through SFT.
- Try to avoid catastrophic forgetting: choose a small LR

Optional Enhancements

- You can use LoRA (or other PEFT Methods) to enhance your model.
- IMPORTANT: LoRA weights count as model weights (max 40M)!

Nest Steps:

1. Split your Group: 4 = 2x2 | 3 = 3
2. Create a SFT Dataset for your alinment training. (Start with Alpaca and add others, if you like.)
3. Start training your Basemodel with SFT
4. Come to us for Leaderboard scores (See excercise factsheet in Canvas for more information.)

Note: If you are not happy with your basemodel you can also revisite pretraining.

Glossary (1/2)

Term	Definition
Adapter Layers	Small bottleneck modules (down-project, nonlinearity, up-project) inserted between transformer layers. An early PEFT method (Houlsby et al., 2019). More parameters than LoRA.
Alignment	The process of steering a pre-trained LLM to follow instructions, be helpful, and avoid harmful outputs. Typically involves SFT, then RLHF or DPO.
Alpaca	Stanford dataset of 52K instruction-response pairs generated by GPT-4 using Self-Instruct. Commonly used for SFT of open models. [3]
Chat Template	Structured text format that marks roles (system, user, assistant) and turn boundaries with special tokens. Examples: ChatML, LLaMA-2 format, Alpaca format.
ChatML	OpenAI's chat markup language. Uses <|im_start|> and <|im_end|> tokens to delimit roles. Clean, extensible, widely adopted.
Full Fine-Tuning	Updating all model parameters during training. For LLaMA-7B: ~42 GB minimum (model + optimizer states). Contrast with PEFT methods.
Instruction Tuning	Synonym for SFT on instruction-response data. Teaches the model to follow instructions rather than simply complete text. [1]

Glossary (2/2)

Term	Definition
LoRA (Low-Rank Adaptation)	PEFT method that freezes W_0 and trains low-rank matrices B (d x r) and A (r x d). Forward: $h = W_0x + BAx$. Merge at inference: no extra latency. [5]
Loss Masking	Setting instruction token labels to -100 so CrossEntropy loss is computed only on response tokens. The single key difference between SFT and standard fine-tuning.
NF4 (NormalFloat 4-bit)	QLoRA's quantization format. Spaces quantization levels according to a normal distribution, reducing error for normally-distributed neural network weights. [6]
PEFT (Parameter-Efficient Fine-Tuning)	Family of methods that freeze the base model and train only small additional components. Includes LoRA, QLoRA, Adapters, Prefix Tuning, Prompt Tuning.
Prefix Tuning	PEFT method that prepends learnable vectors to keys and values in each attention layer. Elegant but can be unstable. (Li and Liang, 2021)
Prompt Tuning	PEFT method that learns soft tokens prepended to the input. Fewest trainable parameters, but limited expressiveness. (Lester et al., 2021)
QLoRA	LoRA combined with NF4 quantization, double quantization, and paged optimizers. Enables LLaMA-65B fine-tuning on a single 48 GB GPU. [6]

Bibliography

[1] Ouyang, L. et al. (2022). Training language models to follow instructions with human feedback. NeurIPS / arXiv:2203.02155.

[2] Raschka, S. (2024). Build a Large Language Model (From Scratch). Manning.

[3] Taori, R. et al. (2023). Stanford Alpaca: An Instruction-following LLaMA model. GitHub.

[4] Wang, Y. et al. (2023). Self-Instruct: Aligning Language Models with Self-Generated Instructions. ACL / arXiv:2212.10560.

[5] Hu, E.J. et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. ICLR / arXiv:2106.09685.

[6] Dettmers, T. et al. (2023). QLoRA: Efficient Finetuning of Quantized Language Models. NeurIPS / arXiv:2305.14314.